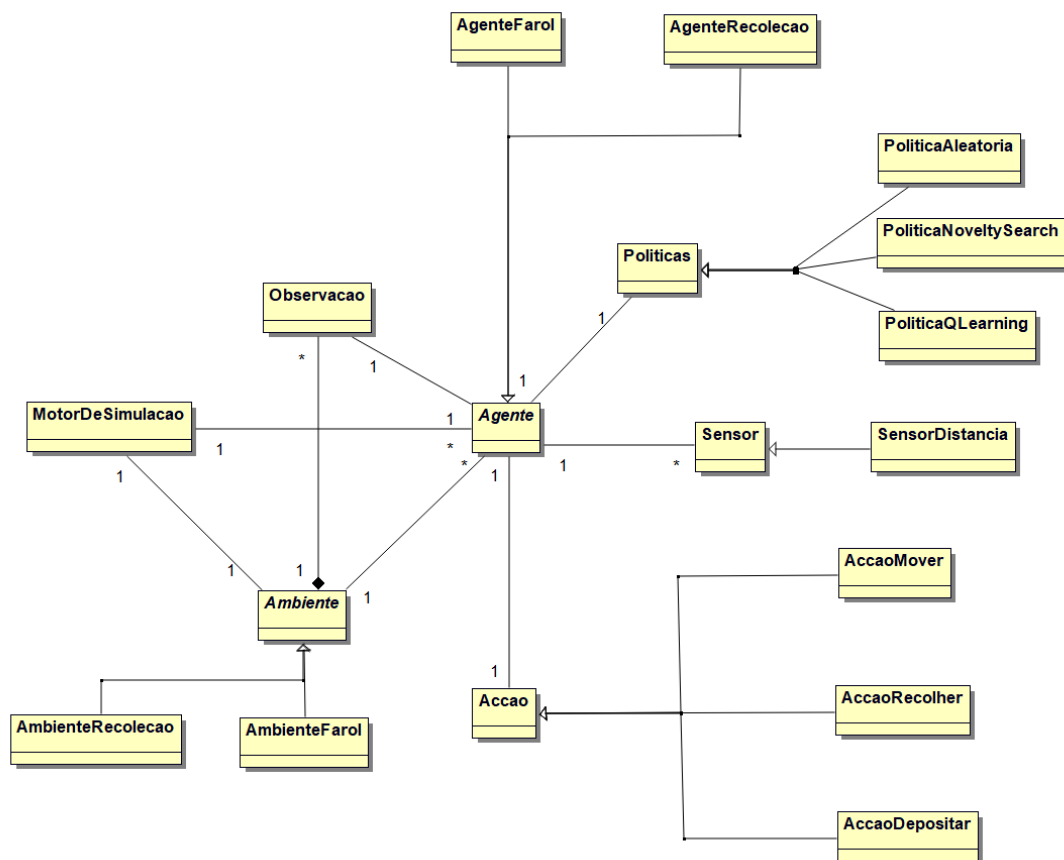


Relatório de planeamento do projeto

Neste projeto ([github link](#)), optámos por implementar os ambientes do Farol e Foraging. Começamos por apresentar aquela que será a estrutura do nosso trabalho, ainda que sujeita a alterações.

Diagrama de classes e divisão dos módulos



De acordo com a nossa interpretação do enunciado, o diagrama de classes organiza-se em três blocos principais: o motor de simulação, os agentes e os ambientes.

O motor de simulação é o “coração” do programa e o módulo onde a execução é inicializada e terminada. Lê o ficheiro com a configuração inicial, inicializa o ambiente e os agentes, e é através dele que recolhemos métricas e avaliamos o comportamento do sistema. É também responsável por gerir o ciclo de simulação, coordenando a receção de observações, a escolha de ações pelos agentes e a aplicação dessas ações no ambiente.

A classe `Ambiente` é abstrata e representa a estrutura do mundo onde os agentes atuam, bem como os elementos que o compõem (agentes, obstáculos, objetivos, itens,

etc.). A partir dela derivam os ambientes concretos que decidimos implementar: AmbienteFarol e AmbienteRecolha, correspondentes a dois dos problemas propostos no enunciado.

A classe Agente é igualmente abstrata e define o comportamento básico e a interface comum a todos os agentes. A partir dela, implementámos duas categorias: o AgenteFarol e o AgenteRecolectao. Cada um destes tem objetivos e métricas diferentes, pelo que achámos necessário separá-los. Associadas aos agentes, temos as Políticas, onde desenvolvemos os algoritmos - neste caso, são três: PolíticaAleatoria, PolíticaNoveltySearch e PolíticaQLearning.

Dividimos os módulos da seguinte forma:

- Agentes – contém todas as classes relacionadas com os agentes, incluindo a observação e os diferentes tipos de agente e os componentes de que necessitam para operar, como ações e sensores. Não utilizámos sincronização, uma vez que o método *executa()* está a controlar todo o ciclo da simulação de forma sequencial e determinística.
- Ambientes: onde estão os ficheiros de cada ambiente, com as suas características;
- Políticas: neste módulo encontram-se as lógicas associadas aos agentes (aleatório, *Q-Learning* e *Novelty Search*).
- Simulador: responsável pela execução da simulação em si. Este módulo coordena o ciclo de vida dos agentes, a interação com o ambiente e o carregamento do ficheiro JSON que define a configuração inicial de cada cenário.

Opções de sensores

Optámos por ter apenas um sensor que permite ao agente ver à sua volta, com um determinado raio. Este sensor dá informação necessária para que o agente consiga distinguir o que o rodeia, sejam obstáculos, recursos ou até mesmo o farol - funciona quase como que um olho humano. No entanto, esta opção ainda está suscetível a mudança, uma vez que ainda estamos a desenvolver o código.

Estratégias e recompensas

Para este trabalho, decidimos implementar três agentes: um agente que se move aleatoriamente e serve apenas como *baseline* para comparação com os agentes que aprendem; dois agentes que aprendem, um por *reinforcement learning*, outro através de um algoritmo evolucionário. Neste contexto, optámos pelo *Q-Learning* e *Novelty Search*, respetivamente.

O *Q-Learning* é uma boa opção porque ajusta-se bem ao tipo de problemas propostos, onde o agente aprende por tentativa-erro e não precisa de conhecer o ambiente em que se insere. Já o *Novelty Search*, conforme abordado em aula, é um algoritmo evolucionário reativo que responde a mudanças no ambiente e procura soluções inesperadas, uma vez que tenta mover-se por caminhos ainda não explorados, evitando ficar preso em máximos ou mínimos locais.

Para analisar o desempenho dos algoritmos, vamos atribuir pontuações a determinadas ações, como por exemplo, à chegada ao farol atribuímos X pontos, ao coletar um recurso, Y pontos. Ainda não definimos a quantidade de pontos adequada para cada recompensa.

- No ambiente do farol:
 - Para o *Q-Learning*, consideramos positivo o facto de o agente chegar ao farol e tomar o menor número de passos possível, bem como não bater em obstáculos, e pontuaremos estas ações - estas serão as recompensas extrínsecas – que são as únicas que importam neste tipo de algoritmo.
 - No *Novelty Search*, a recompensa extrínseca não importa. Pontuaremos o movimento, a descoberta de novos comportamentos e utilizaremos a função *novelty(x)*.
- No ambiente *foraging*:
 - *Q-Learning*: consideramos positivo e atribuiremos pontos aos atos de apanhar recursos, colocá-los no ninho, evitar obstáculos, e a rapidez, ou seja, o número mínimo de passos possível. Estas serão as recompensas extrínsecas.
 - *Novelty Search*: neste algoritmo as recompensas não dependem do ambiente, são apenas intrínsecas, pelo que serão iguais às do ambiente anterior.

Métricas de desempenho

Métricas gerais:

- Número de passos até chegar ao objetivo
- Recompensa em cada simulação
- Número de colisões com obstáculos
- Taxa de sucesso (nº de vezes em que o objetivo foi atingido)

Métricas do ambiente do farol:

- Distância ao farol

Métricas do ambiente *foraging*:

- Número de recursos coletados

- Número de recursos depositados
- Valor total dos recursos
- Eficiência (recursos entregues / passos dados)